THE INDEPENDENT INSTITUTE OF EDUCATION | ROSEBANK INTERNATIONAL COLLEGE

# iTHRIFT CLOTHES

*South African pre-loved branded fashion: website and Android application*

---

XISD6329 Work Integrated Learning 3B

## Task 1: Updated Project Plan, Site Map and Wireframes

| | |
|---|---|
| **Module** | XISD6329 Work Integrated Learning 3B |
| **Assessment** | Task 1 |
| **Group** | 11 Eleven |
| **Group members** | Vukosi Rikhotso (ST10439408) & Theo Golele (ST10439863) |
| **Qualification** | Diploma in Information Technology in Software Development |
| **Institution** | IIE Rosebank College, Pretoria |
| **Client** | iTHRIFT Clothes |
| **Repository** | https://github.com/ST10439408-VRIKHOTSO/iTHRIFT |
| **Date** | 04 September 2026 |

*This submission builds on the project plan, requirement analysis and system design produced for XISD5319 Work Integrated Learning 3A.*

# Table of Contents


1. Update Project Plan ..... 4
   1.1 What has changed since XISD5319 ..... 4
   1.2 The client's needs for the web page and the mobile application ..... 4
      1.2.1 Needs the website must meet ..... 5
      1.2.2 Needs the mobile application must meet ..... 5
      1.2.3 Deciding what belongs on which channel ..... 6
   1.3 Objectives and scope ..... 7
   1.4 Deliverables allocated to each group member ..... 7
   1.5 Updated work breakdown structure ..... 8
   1.6 Updated project schedule ..... 10
   1.7 The DevOps lifecycle ..... 10
   1.8 Updated system architecture ..... 11
   1.9 Updated risk register ..... 15
   1.10 Updated effort and cost budget ..... 16
2. Site Map ..... 18
   2.1 Website site map ..... 18
   2.2 Android application site map ..... 20
   2.3 Website route reference ..... 22
   2.4 Mobile navigation reference ..... 22
   2.5 Differences between the two navigation models ..... 23
3. Wireframes ..... 24
   3.1 Website wireframes ..... 24
   3.2 Android application wireframes ..... 24
   References ..... 35

# List of Figures

Figure 1: Updated twelve-week project schedule with phase bars, owners and the four assessment milestones ..... 12
Figure 2: The eight-phase DevOps lifecycle as applied to the iTHRIFT project, with the tool used at each phase ..... 13
Figure 3: Updated three-tier architecture showing the website and the Android application sharing one REST API and one database ..... 14
Figure 4: Site map of the iTHRIFT website, organised by access tier ..... 19
Figure 5: Site map of the iTHRIFT Android application, with the five bottom-navigation tabs as the top-level nodes ..... 21
Figure 6: Website: catalogue and landing screen (#/shop) ..... 25
Figure 7: Website: product detail with gallery and reviews (#/product/:id) ..... 26
Figure 8: Website: cart with order summary (#/cart) ..... 27
Figure 9: Website: checkout and order confirmation (#/checkout, #/order-confirmation/:id) ..... 28
Figure 10: Website: sign in and registration (#/login, #/register) ..... 29
Figure 11: Website: order history and order tracking (#/orders, #/orders/:id) ..... 30
Figure 12: Website: staff and administrator console (#/admin) ..... 31
Figure 13: Android application: Shop, Search and Product Detail ..... 32
Figure 14: Android application: Cart, Checkout and Order Confirmation ..... 33
Figure 15: Android application: Orders, Order Detail and Account with settings ..... 34

# 1. Update Project Plan

In XISD5319 this group planned, analysed and designed an e-commerce platform for iTHRIFT Clothes, and delivered a working web prototype. XISD6329 takes that work further. The system must now be delivered as two connected products, namely a fully working website and a native Android application. Both of them must be built and released through a DevOps pipeline rather than by hand. This section updates the project plan so that it matches those new requirements.

## 1.1 What has changed since XISD5319

- A second client surface. The solution is no longer only a website. A native Android application, written in Kotlin, now stands beside it and serves customers directly.
- A shared back end. In order to serve two clients from one code base, the application tier is now a JSON REST API. The three-tier structure and the Third Normal Form schema stay the same. Only the contract between the tiers has changed.
- DevOps replaces the hand-over. Testing and deployment used to be phases seven and eight of the old schedule. They are now continuous, because every push to the repository builds and tests both clients before the change is accepted.
- A shorter schedule. The XISD5319 plan ran for fourteen weeks. This module runs for twelve weeks, so the schedule has been rebuilt against the module pacer instead of being adjusted at the edges.

## 1.2 The client's needs for the web page and the mobile application

iTHRIFT Clothes buys pre-owned branded clothing from suppliers, individuals and thrift channels across South Africa. Every item is inspected, photographed and graded as Excellent, Very Good, Good or Fair before it is sold. The business trades from physical stores in Pretoria, and it handles everything else through WhatsApp and Instagram. There is no single system behind any of this.

That informal way of working is the problem. Stock is remembered by one person, prices are quoted in direct messages, and orders are tracked in a chat thread. A customer outside Pretoria therefore has no reliable way to see what is available or to buy it. The business does not lack demand. It lacks a channel that can carry the demand it already has.

The fourteen use cases recorded in the XISD5319 requirement analysis have not changed. What has changed is that they must now be met on two clients that do very different jobs. Staff work at a desk with a keyboard, a large screen and a folder of product photographs to upload, which is a website task. Customers in South Africa shop on a phone. If the mobile application were only a shrunken copy of the website, staff would be given a console they cannot use, and customers would be given a store that feels like a website squeezed into a frame. The two channels are therefore specified separately below.

### 1.2.1 Needs the website must meet

The website is the full-service channel. It is the only surface on which the business itself operates, and it is the surface a customer reaches from a shared link, a search result or a desktop computer at work.

Table 1: Client needs served by the website

| Client need | How the website meets it |
|---|---|
| A branded storefront that shows stock properly | Catalogue grid with large photographs, brand, size, condition grade and price on every card, and a detail page with a full image gallery |
| Customers must find a specific piece quickly | Keyword search plus filters on category, brand, condition, price range and stock, with sorting by price and by newest |
| Customer accounts and order history | Registration, sign-in, profile and delivery address management, and a full order history |
| Buying without a phone call | Cart, checkout with a delivery address, PayFast card payment or EFT, and an order confirmation with a reference number |
| Customers can see where their order is | Status tracker moving through Placed, Processing, Shipped and Delivered, with the courier reference shown once captured |
| Staff must manage listings without a developer | Admin console with add, edit, mark out of stock and delete for every listing, including photograph upload and condition grading |
| Staff must process orders as they arrive | Order queue filtered by status, with a status update control and a field for the courier reference |
| Management need to see how the business is doing | Dashboard with revenue, order counts by status, revenue by brand and by category, and a low-stock report |
| Administrators must control who has access | User administration to view, suspend, reactivate or delete customer accounts, visible to the administrator role only |
| Buyers trust other buyers | Star rating and written reviews on the product detail page, restricted to signed-in customers |

### 1.2.2 Needs the mobile application must meet

The Android application is the customer channel. It carries no administrative functions at all. Its purpose is to make browsing and buying fast on a phone, and to be the place where a returning customer checks an order without opening a browser.

Table 2: Client needs served by the Android application

| Client need | How the application meets it |
|---|---|
| Reach customers where they already are | An installed Android application with an icon on the home screen, rather than a link the customer has to remember |
| Browsing must feel quick on a phone | Two-column product grid, category chips across the top, and images cached so a scroll back up does not re-download |
| Search that suits a small screen | A dedicated Search tab with the filter panel on its own screen rather than a dropdown squeezed into a header |

| Client need | How the application meets it |
|---|---|
| Signing in without a long password on a phone keyboard | Email and password sign-in with the password encrypted in transit and at rest, plus single sign-on so a customer can use an existing account |
| Buy in as few taps as possible | Add to cart from the product detail screen, a persistent cart badge on the bottom navigation, and a three-step checkout |
| Check an order without opening a browser | Orders tab with a status badge on every order and a vertical status tracker on the order detail screen |
| Control over the application itself | Settings screen covering the API endpoint, notification preference, theme and local cache, and a clear sign-out |
| The application must not break on a bad connection | Every network call returns a typed result, and each screen has a defined loading, empty and error state rather than a crash |

### 1.2.3 Deciding what belongs on which channel

Table 3 records the decision made for every function, together with the reason for it. The table also states clearly what the application will not do.

Table 3: Allocation of functions to the website and the Android application

| Function | Web | App | Reason for the decision |
|---|---|---|---|
| Browse and filter the catalogue | Yes | Yes | Core to both; the reason a customer opens either |
| Product detail and reviews | Yes | Yes | Same content, laid out for the available width |
| Register, sign in, manage profile | Yes | Yes | A customer must be able to start on one and continue on the other |
| Cart, checkout and payment | Yes | Yes | Both are selling channels |
| Order history and tracking | Yes | Yes | The most common reason a customer returns |
| Manage listings | Yes | No | Staff upload photographs from a computer and work from a large screen |
| Process orders, capture courier references | Yes | No | Done at a desk against a printed picking list |
| Reports and dashboard | Yes | No | Tables and charts are unreadable on a phone |
| User administration | Yes | No | Administrator-only and low frequency; no case for a mobile version |
| Push notification on status change | No | Yes | Only the installed application can reach the customer once it is closed |

## 1.3 Objectives and scope

The objective is to deliver, test and release a working two-channel solution for iTHRIFT Clothes within twelve weeks. The solution must include a fully working website that covers both the storefront and the staff console. It must include a native Android application that runs on a physical handset and not only on an emulator. One REST API and one Third Normal Form database must serve both clients. The full DevOps lifecycle must be followed, using an Azure Boards Kanban board and GitHub Actions, with automated tests that run on every push. Finally, the application must be prepared for publication on the Google Play Store.

Table 4: Project scope for XISD6329

| In scope | Out of scope for this phase |
|---|---|
| Customer storefront on web and Android | A marketplace where customers list their own items |
| Staff and administrator console on the web | An iOS application |
| Shared REST API and relational database | Multi-currency pricing and international delivery |
| Authentication, roles and single sign-on | A loyalty or points scheme |
| Cart, checkout, payment capture, order lifecycle | Live integration with a courier tracking API |
| Reviews and ratings | In-app live chat with the shop |
| Reporting for management | Automated accounting or stock-reorder integration |
| Automated build, test and release pipeline | Production hosting on a paid commercial plan |
| Help system and user documentation | Formal penetration testing by a third party |

## 1.4 Deliverables allocated to each group member

Roles carry over from XISD5319, but the split has been adjusted so that each member owns one client surface from start to finish, while the shared API and database remain jointly owned. Ownership means that the named member is accountable for finishing the deliverable and for making it work. It does not prevent the other member from contributing.

Table 5: Deliverables allocated to each group member

| Deliverable | Owner | Supporting member | Due |
|---|---|---|---|
| Updated project plan, scope and risk register | Theo Golele | Vukosi Rikhotso | Week 3 |
| Site maps for the website and the application | Vukosi Rikhotso | Theo Golele | Week 3 |
| Website wireframes | Theo Golele | Vukosi Rikhotso | Week 4 |
| Android application wireframes | Theo Golele | Vukosi Rikhotso | Week 4 |
| GitHub repository, branch policy and README | Vukosi Rikhotso | Theo Golele | Week 2 |
| Azure DevOps project and Kanban board | Vukosi Rikhotso | Theo Golele | Week 3 |
| Task 1 document, compiled and submitted | Theo Golele | Vukosi Rikhotso | Week 5 |
| Website markup, styling and client-side logic | Theo Golele | Vukosi Rikhotso | Week 7 |
| REST API endpoints and business rules | Vukosi Rikhotso | Theo Golele | Week 8 |
| Database schema, seed data and migrations | Vukosi Rikhotso | None | Week 6 |

| Deliverable | Owner | Supporting member | Due |
|---|---|---|---|
| Android UI: Shop, Search and Product Detail | Theo Golele | Vukosi Rikhotso | Week 8 |
| Android UI: Cart, Checkout, Orders and Account | Vukosi Rikhotso | Theo Golele | Week 9 |
| Authentication, roles and single sign-on | Vukosi Rikhotso | Theo Golele | Week 8 |
| Unit tests and GitHub Actions workflows | Vukosi Rikhotso | Theo Golele | Week 10 |
| Help system and user documentation | Theo Golele | None | Week 11 |
| Demonstration video and release notes | Theo Golele | Vukosi Rikhotso | Week 11 |
| Signed release build and Play Store assets | Vukosi Rikhotso | Theo Golele | Week 11 |
| Final project report and presentation | Both members | None | Week 12 |

Accountability is enforced through the Kanban board. Every work item has an assignee, and any item that has not moved for a week is raised at the weekly collaboration session.

### 1.5 Updated work breakdown structure

The structure below replaces the eight phases used in XISD5319. It now has eleven phases, because the client tier has been split into two products and because testing and release have moved from end-of-project phases to continuous activities.

Table 6: Updated work breakdown structure

| WBS | Work package | Dur. | Pred. | Owner |
|---|---|---|---|---|
| P1 | Project initiation and re-planning | 2 wk | None | Both |
| 1.1 | Review XISD5319 feedback and confirm what carries over | 2 d | None | Both |
| 1.2 | Confirm the two-channel scope with the client | 1 d | 1.1 | Theo |
| 1.3 | Rebuild the schedule against the module pacer | 2 d | 1.2 | Theo |
| 1.4 | Update the risk register and the effort budget | 2 d | 1.3 | Both |
| P2 | Design update: site map and wireframes | 4 wk | P1 | Both |
| 2.1 | Derive the site map from the implemented routes | 3 d | 1.2 | Vukosi |
| 2.2 | Draw the website wireframes | 4 d | 2.1 | Theo |
| 2.3 | Draw the Android application wireframes | 4 d | 2.1 | Theo |
| 2.4 | Review the wireframes against the fourteen use cases | 2 d | 2.2, 2.3 | Both |
| 2.5 | Compile and submit the Task 1 document | 2 d | 2.4 | Theo |
| P3 | DevOps set-up | 2 wk | P1 | Vukosi |
| 3.1 | Create the GitHub repository and branch policy | 1 d | 1.1 | Vukosi |
| 3.2 | Create the Azure DevOps project and Kanban board | 1 d | 3.1 | Vukosi |
| 3.3 | Load the backlog and link work items to branches | 2 d | 3.2 | Both |
| 3.4 | Write the first build and test workflows | 3 d | 3.1 | Vukosi |

| WBS | Work package | Dur. | Pred. | Owner |
|---|---|---|---|---|
| P4 | Website update and refactor | 5 wk | P2 | Theo |
| 4.1 | Rework the storefront against the wireframes | 5 d | 2.2 | Theo |
| 4.2 | Catalogue, search, filter and product detail pages | 5 d | 4.1 | Theo |
| 4.3 | Cart, checkout and order tracking pages | 4 d | 4.2 | Theo |
| 4.4 | Staff and administrator console | 5 d | 4.3 | Theo |
| 4.5 | Accessibility and responsive behaviour pass | 3 d | 4.4 | Theo |
| P5 | REST API and database | 5 wk | P2 | Vukosi |
| 5.1 | Confirm the API contract used by both clients | 2 d | 2.1 | Both |
| 5.2 | Schema, seed data and migration scripts | 4 d | 5.1 | Vukosi |
| 5.3 | Authentication, roles and single sign-on | 5 d | 5.2 | Vukosi |
| 5.4 | Catalogue, cart and order endpoints | 6 d | 5.2 | Vukosi |
| 5.5 | Reporting endpoints for the dashboard | 3 d | 5.4 | Vukosi |
| P6 | Android application development | 6 wk | P5 | Both |
| 6.1 | Project skeleton, navigation shell and theme | 3 d | 5.1 | Theo |
| 6.2 | API client, session handling and typed results | 4 d | 6.1 | Vukosi |
| 6.3 | Shop, Search and Product Detail screens | 6 d | 6.2 | Theo |
| 6.4 | Cart, Checkout and Order Confirmation screens | 6 d | 6.3 | Vukosi |
| 6.5 | Orders, Order Detail and Account screens | 5 d | 6.4 | Vukosi |
| 6.6 | Install and test on a physical handset | 3 d | 6.5 | Both |
| P7 | Automated testing | 5 wk | P3 | Vukosi |
| 7.1 | Unit tests for the API business rules | 4 d | 5.4 | Vukosi |
| 7.2 | End-to-end smoke test across the API surface | 3 d | 7.1 | Vukosi |
| 7.3 | Instrumented and unit tests for the application | 4 d | 6.5 | Both |
| 7.4 | Wire the test suites into GitHub Actions | 2 d | 7.2, 7.3 | Vukosi |
| P8 | Help system and user documentation | 4 wk | P4 | Theo |
| P9 | Release preparation | 3 wk | P6, P7 | Both |
| 9.1 | Signed release build and release notes | 3 d | 7.4 | Vukosi |
| 9.2 | Play Store listing, screenshots and assets | 3 d | 9.1 | Vukosi |
| 9.3 | Demonstration video with voice-over | 3 d | 6.6 | Theo |
| P10 | Final project report | 2 wk | P9 | Both |
| P11 | Presentation and hand-over | 1 wk | P10 | Both |

## 1.6 Updated project schedule

Figure 1 shows the twelve-week schedule. The critical path runs from P1 to P2 to P5 to P6 to P7 to P9 to P10. The API must exist before the Android client can be built against it, the client must exist before it can be tested, and the tests must pass before a release can be prepared. Website work is deliberately kept off the critical path, so a delay there costs presentation quality rather than the submission itself.

Weeks 5 to 7 carry three phases at the same time. This is only possible because the API contract is agreed in work package 5.1 before any client code is written. Both developers then build against the same agreed contract instead of waiting for each other.

## 1.7 The DevOps lifecycle

DevOps joins development and operations into one continuous loop, so that software is built, tested and released repeatedly and reliably instead of in one large hand-over at the end. Figure 2 shows the eight phases, and Table 7 records the tool used at each phase, together with what the group actually does there.

Table 7: The eight DevOps phases as applied to this project

| Phase | Tool | What happens |
|---|---|---|
| Plan | Azure Boards Kanban | Work packages become epics and are then broken into items that are small enough to finish in a day or two. Each item names an assignee and an acceptance test, and each item is linked to the branch that implements it. |
| Code | GitHub, VS Code, Android Studio | One repository holds the website, the API and the application, so a change to a shared contract is made in a single commit. Work happens on short-lived feature branches. The main branch is protected and changes only through a reviewed pull request. |
| Build | GitHub Actions, Gradle | Two workflows run on every push. One workflow installs the API from the lock file and starts the server, and the other assembles a debug build of the application. This proves that the code builds on a clean machine and not only on our own computers. |
| Test | JUnit, Node smoke test | An end-to-end suite tests the API in the same way that a real client would use it, covering registration, sign-in, filtering, cart, checkout, stock decrement, order status and the role checks. Unit tests cover the pure logic of the application. A failed pipeline blocks the merge. |
| Release | Signed package artefact | A build that passes produces a versioned artefact with release notes that list what has changed since the previous build. |
| Deploy | Google Play Console | The package is installed on a physical handset for the demonstration and, at the end of the module, uploaded to the Play Console. The API is deployed to a host that both clients can reach. |
| Operate | Demonstration environment | A seeded database, known demonstration accounts for the customer, staff and administrator roles, and the help system reachable from within the application. |

| Phase | Tool | What happens |
|---|---|---|
| Monitor | Server logging, Logcat | Structured logging records every request and every error, and Android logging records the boundaries of each network call. Anything that is found becomes a new work item, which closes the loop back to Plan. |

To keep the lifecycle honest, the group has agreed on one definition of done. A work item is finished when it has been merged through a reviewed pull request, when both build workflows pass, when tests cover the new behaviour and the whole suite passes, when any user-facing text or README section affected has been updated, when the change has been tested by hand on a handset or in a browser, and when the item has been moved to Done with a note on anything that is still outstanding.

## 1.8 Updated system architecture

The three-tier architecture specified in the XISD5319 system design is kept. The change is that the presentation tier now has two occupants, and that the contract between the presentation tier and the application tier is JSON over HTTPS rather than server-rendered HTML. Figure 3 shows the result.

The security reasoning behind the original design has not changed, and it is now even more important. The database can be reached only from the application tier over the local interface, every query is parameterised, and role checks are carried out on the server for every request. Hiding a menu item in either client is a convenience only, and never a control.

The three figures for this section follow on the next three pages. They are the schedule, the DevOps lifecycle and the architecture.

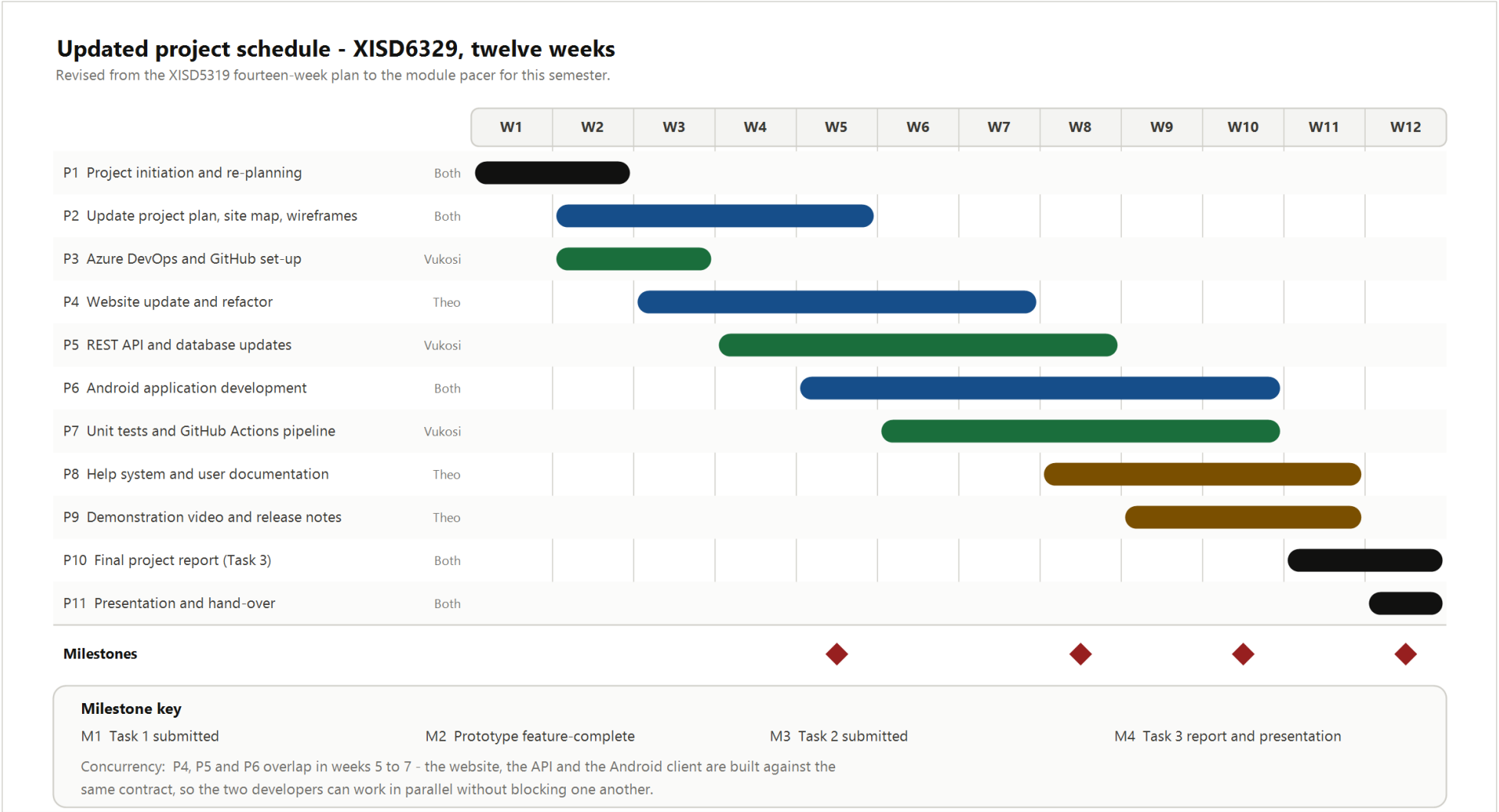



Figure 1: Updated twelve-week project schedule with phase bars, owners and the four assessment milestones

- M1 in week 5 is this Task 1 submission, M3 in week 10 is Task 2, which is the working prototype, and M4 in week 12 is the final report and presentation.
- M2 in week 8 is an internal milestone rather than an assessment point. It is the date by which every feature must exist in some form, which leaves two weeks for repairing defects instead of adding new work.
- Bar colour shows the owner named in the column to the left of the chart. Black is used for joint work, blue for Theo Golele, green for Vukosi Rikhotso and amber for documentation deliverables.

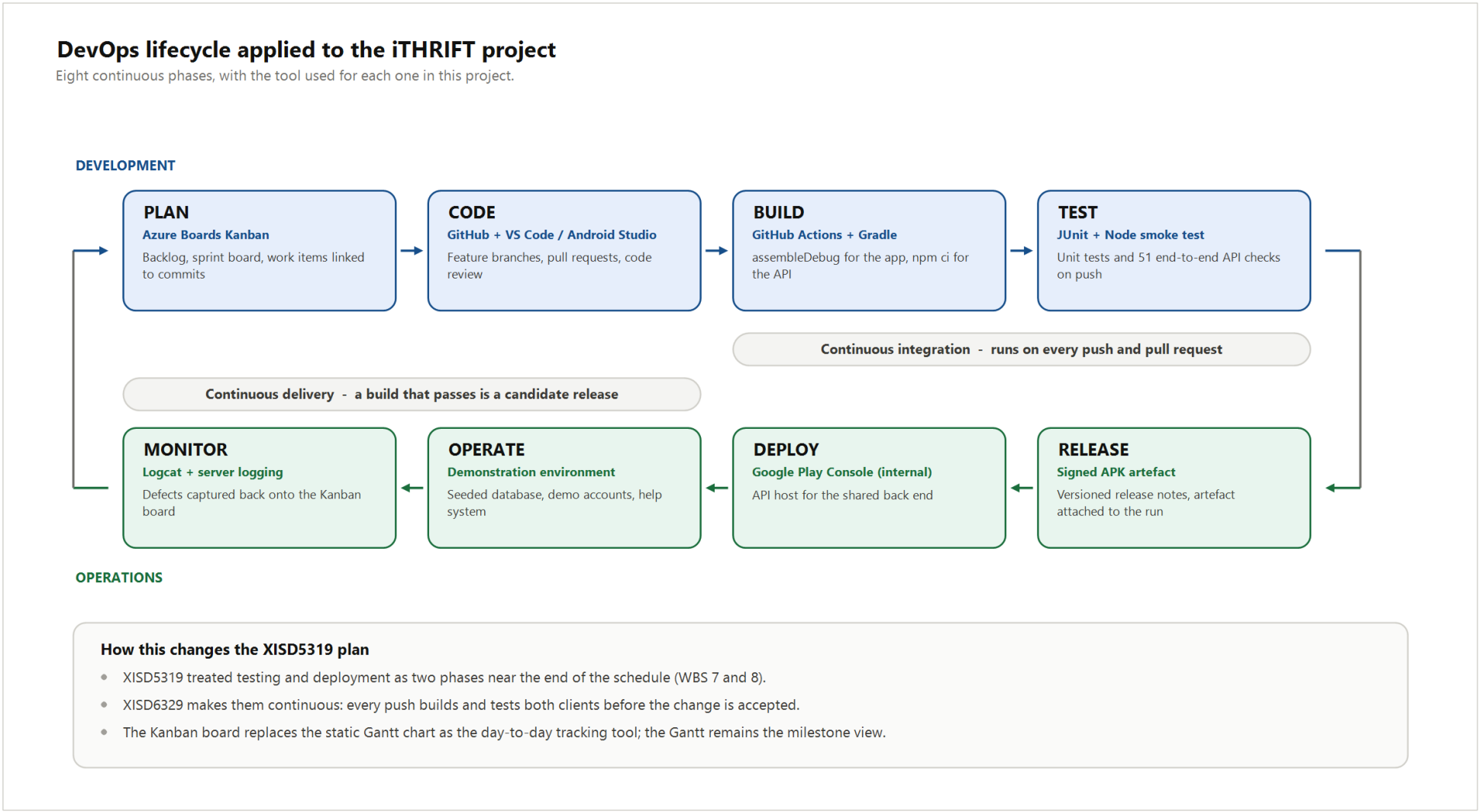



Figure 2: The eight-phase DevOps lifecycle as applied to the iTHRIFT project, with the tool used at each phase

- The upper row is the development loop and the lower row is the operations loop. The two rows connect at each end, and that is what makes the lifecycle continuous rather than sequential.
- Continuous integration covers Build and Test. No change is merged until both clients build successfully and every test passes.
- Continuous delivery covers Release and Deploy. Any build that clears integration is a possible release, which means that releasing becomes a decision rather than a rush.

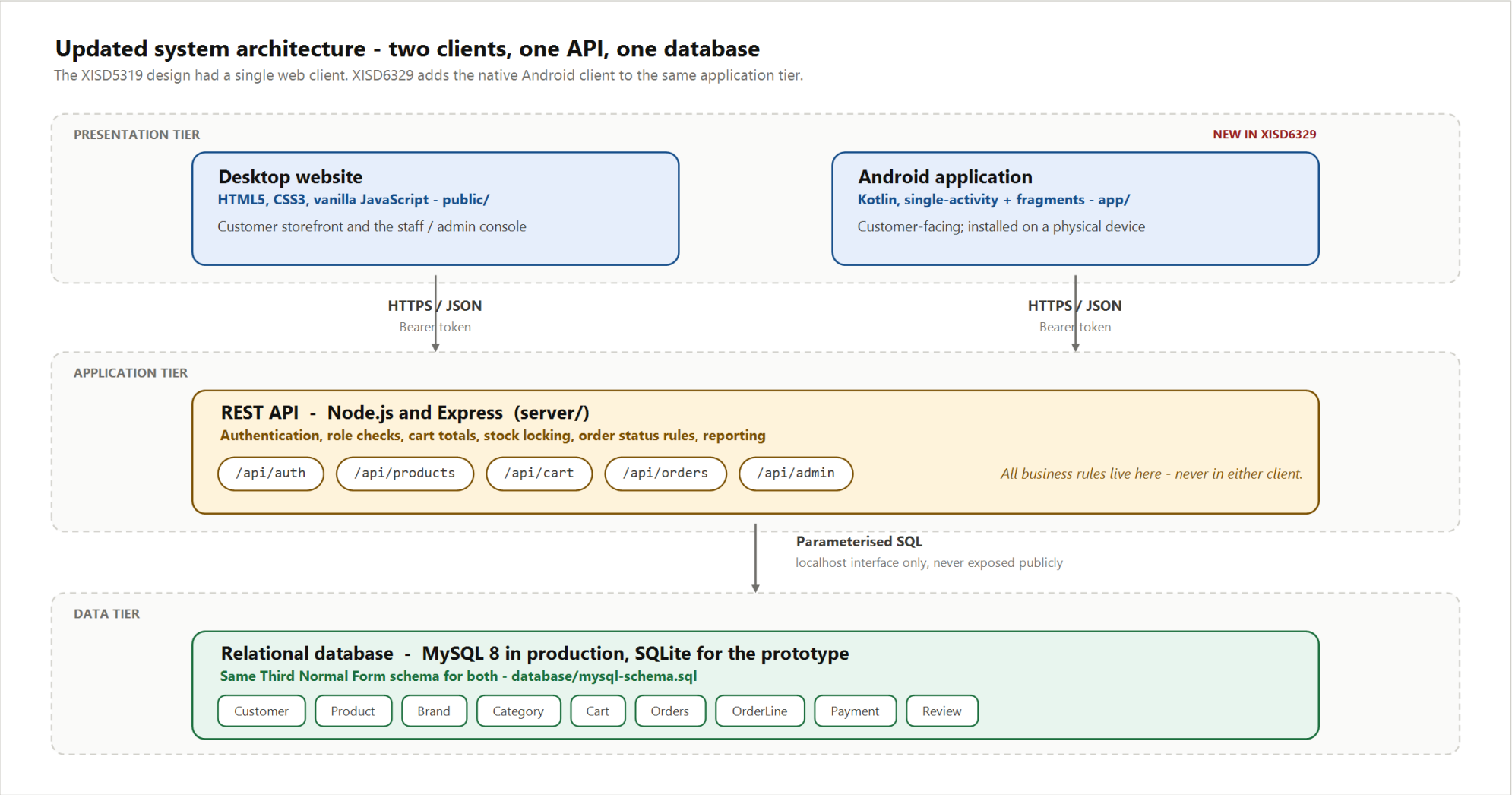



Figure 3: Updated three-tier architecture showing the website and the Android application sharing one REST API and one database

- Both clients authenticate in the same way and carry a bearer token on every request that follows, so a customer's account works in exactly the same way on either surface.
- All the business rules live in the application tier. These rules include cart totals, stock locking at checkout, order status transitions and role permissions. Neither client is trusted to enforce them.
- The prototype runs against SQLite so that it stays portable during development, while the production schema is MySQL 8. The table structure is identical, which is why the same queries serve both.

## 1.9 Updated risk register

Probability and impact are each rated from 1 to 5. The product of the two gives a score, which is banded as Low (1 to 6), Medium (8 to 12) or High (15 to 25). Three risks are new this semester and are marked as such.

Table 8: Updated risk register

| ID | Risk | P | I | Score | Mitigation and contingency | Owner |
|---|---|---|---|---|---|---|
| R1 | A two-member group loses a member to illness or workload | 3 | 5 | 15 High | Every deliverable has a named supporting member who can take it over. Contingency: reduce the scope of the reporting dashboard first, because it is the least assessed feature. | Both |
| R2 | The application will not run on a physical handset (new) | 3 | 5 | 15 High | Install on a real device from week 6 rather than week 11, and repeat this at the end of every sprint. Contingency: borrow a handset and record the demonstration video early. | Theo |
| R3 | The API contract drifts and the clients disagree (new) | 4 | 3 | 12 Med | The contract is agreed in work package 5.1; any change requires a pull request updating both clients together. Contingency: the smoke test fails loudly on a contract break. | Vukosi |

| ID | Risk | P | I | Score | Mitigation and contingency | Owner |
|---|---|---|---|---|---|---|
| R4 | PayFast integration proves harder than expected | 3 | 3 | 9 Med | Treat the live gateway as a stretch goal and build against a simulated payment capture. Contingency: demonstrate EFT and cash on delivery, which need no third party. | Vukosi |
| R5 | The build pipeline itself becomes a blocker (new) | 3 | 3 | 9 Med | Keep the workflows small and cache dependencies; a workflow broken for more than a day is fixed before feature work continues. Contingency: run the same commands locally and record the output. | Vukosi |
| R6 | Scope creep from features that are interesting but unassessed | 4 | 2 | 8 Med | Anything not in Table 4 goes to a Later column rather than into the sprint. Contingency: the weekly session with the lecturer confirms priorities. | Both |
| R7 | Slow or absent client feedback delays a decision | 3 | 2 | 6 Low | Recommendations are put in writing, with a stated default that applies if no answer arrives within five working days. Contingency: proceed on the default and record it. | Theo |
| R8 | Customer personal data is exposed | 2 | 5 | 10 Med | Passwords are hashed, transport is over HTTPS, roles are checked on the server, and no personal data is placed in a URL. Contingency: seed the demonstration database with fictitious customers only. | Vukosi |
| R9 | A third-party service is unavailable during the demonstration | 2 | 4 | 8 Med | Record the demonstration video in advance and seed the database, so that the walk-through needs no live external call. Contingency: present from the recording. | Theo |
| R10 | Gaps in the team's experience with Kotlin or with Actions | 3 | 3 | 9 Med | Spike work items are limited to one day. If a spike runs over that limit, the group asks the lecturer in the weekly session instead of continuing alone. | Both |

After mitigation, R1 and R2 remain in the High band. Both are handled by the same discipline, which is that nothing is left until the end. Physical-device testing and cross-cover on every deliverable are scheduled from week 6, which is early enough for a problem to be inconvenient rather than fatal.

## 1.10 Updated effort and cost budget

All the tooling is open source or on a free tier, so there are no licence or hardware costs. The budget is therefore an effort budget, costed at a notional junior developer rate of R150 per hour. The module allocates 150 notional hours per student, which gives 300 hours for two students. The plan commits 280 hours, which leaves 20 hours of contingency.

Table 9: Updated effort and cost budget

| Phase | Theo (h) | Vukosi (h) | Total (h) | Cost at R150/h |
|---|---|---|---|---|
| P1 Initiation and re-planning | 12 | 10 | 22 | R3 300 |
| P2 Site map and wireframes | 26 | 14 | 40 | R6 000 |
| P3 DevOps set-up | 6 | 16 | 22 | R3 300 |
| P4 Website update and refactor | 38 | 6 | 44 | R6 600 |
| P5 REST API and database | 6 | 40 | 46 | R6 900 |
| P6 Android application | 26 | 30 | 56 | R8 400 |
| P7 Automated testing | 8 | 20 | 28 | R4 200 |
| P8 Help system and documentation | 14 | 2 | 16 | R2 400 |
| P9 Release preparation | 10 | 10 | 20 | R3 000 |
| P10 and P11 Report and presentation | 14 | 12 | 26 | R3 900 |
| Total committed | 160 | 160 | 280 | R42 000 |
| Contingency (hours not committed) | Not allocated | Not allocated | 20 | R3 000 |

The committed effort comes to R42 000, which is the same total as the XISD5319 budget. This is a useful check, because it shows that the work has been redistributed across more phases rather than expanded. In return for that cost, the client receives two working sales channels, an administrative console that removes manual stock tracking, and a release pipeline that makes future changes cheap.

# 2. Site Map

The site maps below were not drawn from imagination. Each node matches a route that the front-end router resolves, or a fragment that the Android activity hosts. Because they were produced in this way, they also serve as a specification for Task 2. A route on the map that does not exist in the build is an outstanding work item, and a route in the build that is not on the map is scope that was never agreed. Both maps are organised according to who is allowed to reach a screen, because access is the property that most often goes wrong in an e-commerce system.

## 2.1 Website site map

The website has three access tiers. Anyone may browse the catalogue and read reviews. A signed-in customer may also reach the cart, checkout, order history and account screens. Staff and administrators reach the console, and within the console the Customers section is limited to administrators only. The map is drawn in Figure 4, and every route is listed in Table 10.

The two maps are placed on the following two pages so that they can be compared directly. The reference tables then follow.

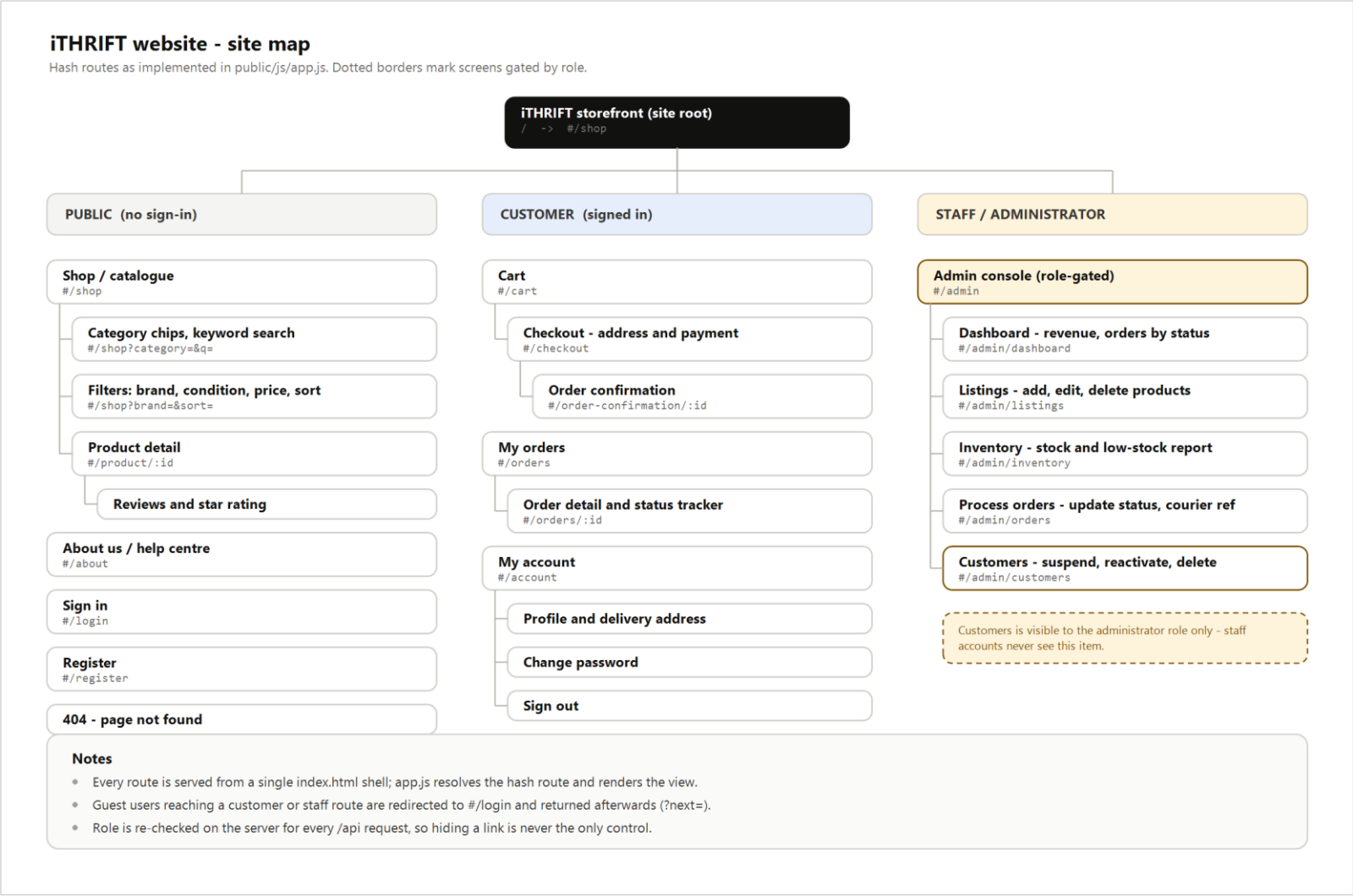



Figure 4: Site map of the iTHRIFT website, organised by access tier

- The site is a single-page application. One HTML shell is served, and the router resolves the hash route and renders the matching view. The routes shown are real addresses that can be linked to and bookmarked.
- A guest who follows a link to a customer or staff route is sent to the sign-in screen and returned to the original destination afterwards, instead of being shown an error.
- The role is checked again on the server for every API request behind these screens. The site map shows what a user can see, but it is not what enforces access.

## 2.2 Android application site map

The application uses a single-activity design. One activity hosts a fragment container and a bottom navigation bar with five tabs, and every other screen opens on top of the current tab through the fragment back stack. This keeps the navigation model shallow, which matters on a phone, because the customer is never more than two taps away from the catalogue. The map is drawn in Figure 5, and the screens are listed in Table 11.

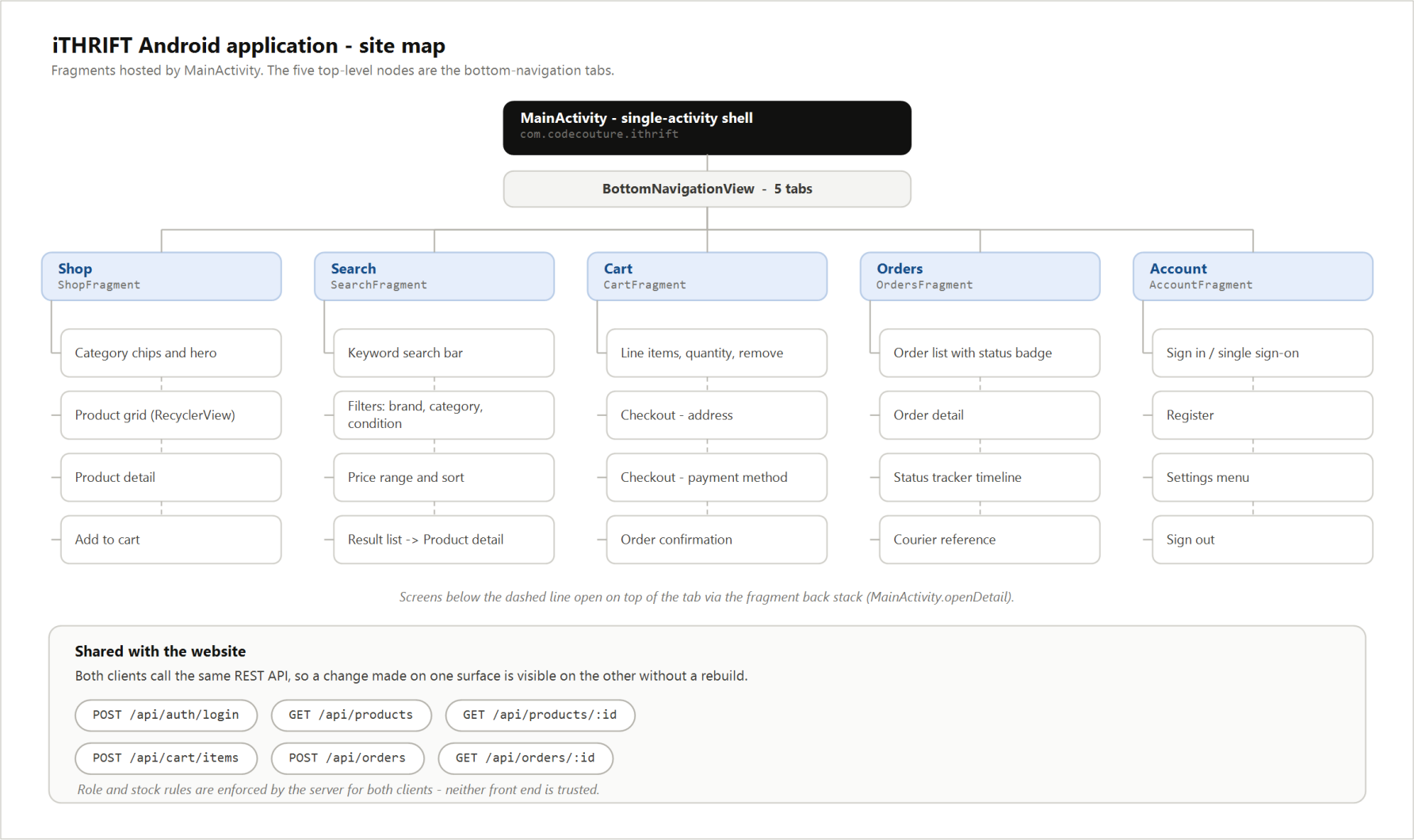



Figure 5: Site map of the iTHRIFT Android application, with the five bottom-navigation tabs as the top-level nodes

- The five tabs are the only top-level destinations. Screens below the dashed line are detail screens pushed onto the back stack, so the system back button always returns the customer to where they came from.
- The application carries no administrative screens at all, which is the decision recorded in Table 3.
- The endpoints listed at the foot of the figure are shared with the website, so a change to a business rule takes effect on both surfaces without either one being rebuilt.

## 2.3 Website route reference

Table 10: Website routes, access level and purpose

| Route | Access | Purpose |
|---|---|---|
| #/shop | Public | Catalogue with category chips, keyword search and filters |
| #/product/:id | Public | Product detail, image gallery, condition and stock, reviews |
| #/about | Public | About the business, condition grading, delivery and returns |
| #/login | Public | Sign in, with single sign-on options |
| #/register | Public | Create a customer account |
| #/cart | Customer | Cart line items, quantity, running total |
| #/checkout | Customer | Delivery address and payment method |
| #/order-confirmation/:id | Customer | Order reference, total and delivery estimate |
| #/orders | Customer | Order history with a status badge on each order |
| #/orders/:id | Customer | Order detail, status tracker and courier reference |
| #/account | Customer | Profile, delivery address, password, sign out |
| #/admin/dashboard | Staff | Revenue, order counts, revenue by brand and category |
| #/admin/listings | Staff | Add, edit, mark out of stock and delete listings |
| #/admin/inventory | Staff | Stock levels and the low-stock report |
| #/admin/orders | Staff | Order queue, status updates, courier reference capture |
| #/admin/customers | Admin | View, suspend, reactivate and delete customer accounts |

## 2.4 Mobile navigation reference

Table 11: Android application screens, host fragment and how each is reached

| Screen | Fragment | Reached from |
|---|---|---|
| Shop | ShopFragment | Bottom navigation, and the landing screen on launch |
| Search and filters | SearchFragment | Bottom navigation |
| Product detail | ProductDetailFragment | Product card in Shop or in a search result |
| Cart | CartFragment | Bottom navigation, or after adding an item |
| Checkout | CheckoutFragment | Proceed to checkout, from the Cart |
| Order confirmation | OrderConfirmationFragment | Confirm and pay, from Checkout |
| Orders | OrdersFragment | Bottom navigation, or Track order from confirmation |
| Order detail and tracker | OrderDetailFragment | An order row in Orders |
| Account, sign in and settings | AccountFragment | Bottom navigation, or the header sign-in button |

## 2.5 Differences between the two navigation models

The two maps are deliberately not identical, and the differences between them are design decisions rather than accidents.

- Search is a tab on the phone and a control on the website. On a large screen the filter panel can sit beside the results, but on a phone it needs the whole screen, so it earns its own destination.
- Sign-in is a page on the website and a section of the Account tab on the phone. A phone user expects the Account tab to handle identity, while a web user expects a sign-in page that can be linked to.
- The website exposes checkout as its own route so that the customer can be returned to it after a payment redirect. The application treats checkout as a screen on the cart stack, because there is no redirect to return from.
- The console exists only on the website. This is the largest structural difference, and it follows directly from Table 3.

# 3. Wireframes

The wireframes are low fidelity on purpose. They fix the layout, the hierarchy and the position of every control, and they deliberately do not fix the final photography, wording or colour, which are settled during the build. The conventions below apply throughout.

- A grey rectangle crossed with two diagonals is an image placeholder. The label inside it states what the image is.
- Grey bars stand for body copy whose exact wording is not yet fixed.
- A solid dark pill is the primary action on that screen, and an outlined pill is a secondary action. There is never more than one primary action visible at a time.
- A numbered amber marker is a note about behaviour that the drawing alone cannot show.
- Placeholder values are written as R000 for money and ORD-0000 for references, so that no wireframe can be mistaken for real data.

## 3.1 Website wireframes

Seven screens are wireframed, in Figures 6 to 12. Together they cover every route in Table 10, apart from the About page, which is static content, and the page-not-found fallback.

## 3.2 Android application wireframes

Nine application screens are wireframed across three sheets, in Figures 13 to 15, and they are grouped by the task that the customer is performing. All of them are drawn at a viewport of 360 by 760 density-independent pixels, which is representative of the mid-range Android handsets that are common in South Africa.

The full set follows on the next ten pages, with the website screens first and the application screens after them. Each sheet is placed on its own landscape page so that the labels stay readable, and the design decisions behind that screen are listed beneath it. The image files are also stored in the repository under docs/task1/figures, in case any detail needs to be inspected at full size.

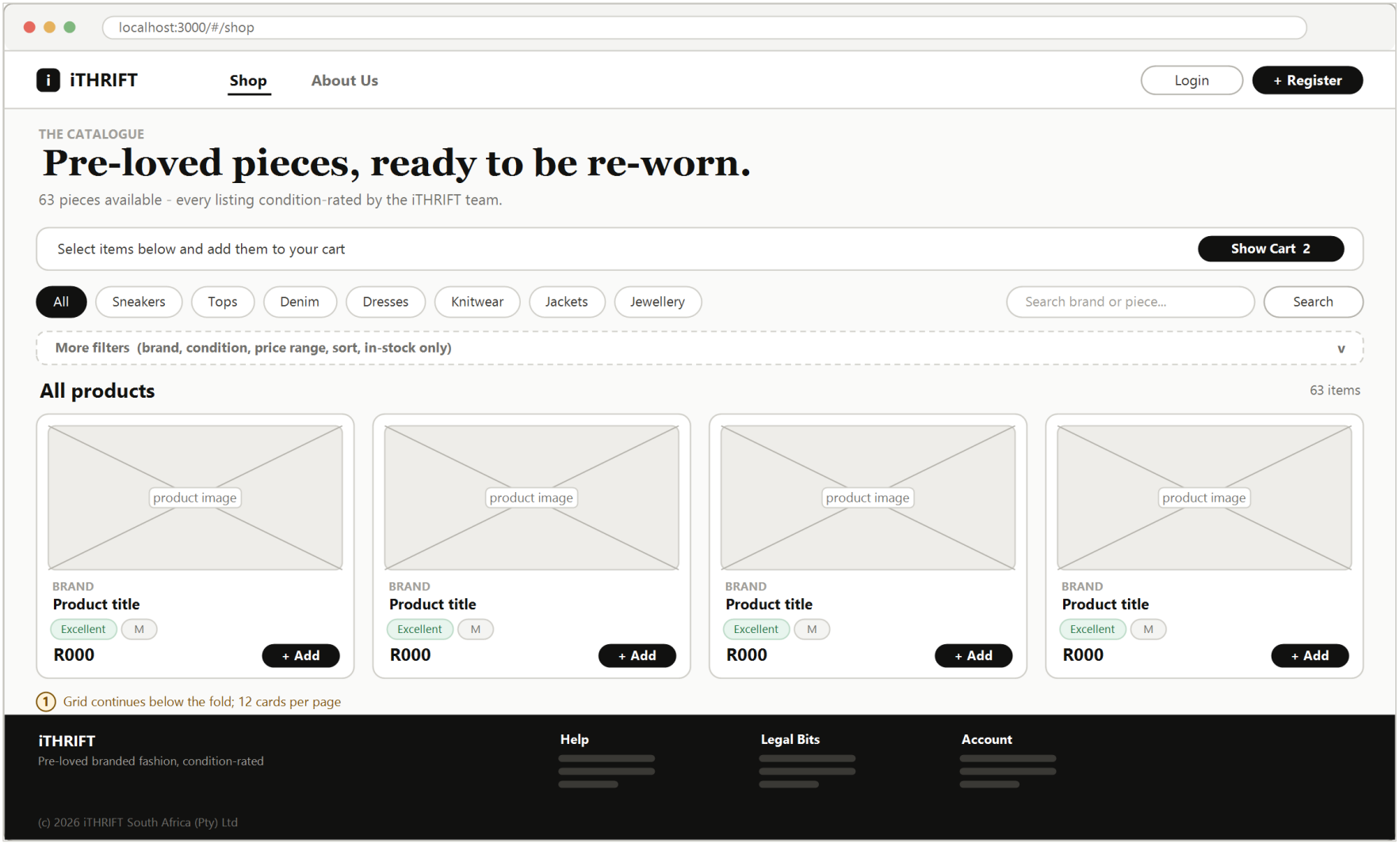



Figure 6: Website: catalogue and landing screen (#/shop)

- The catalogue is the landing screen. A visitor sees the stock before being asked to do anything, which is the single most important decision on the site.
- Filtering is layered. Category chips are always visible because they carry most of the traffic, while brand, condition, price and sort are collapsed behind More filters, so that the controls do not crowd the products.
- Every card carries the condition grade. Condition is the deciding factor when buying second-hand clothing, so it is shown at the point of choice rather than one click away.
- Add is available directly from the card. A visitor who is not signed in is sent to sign-in and then returned to the catalogue with the item still selected.

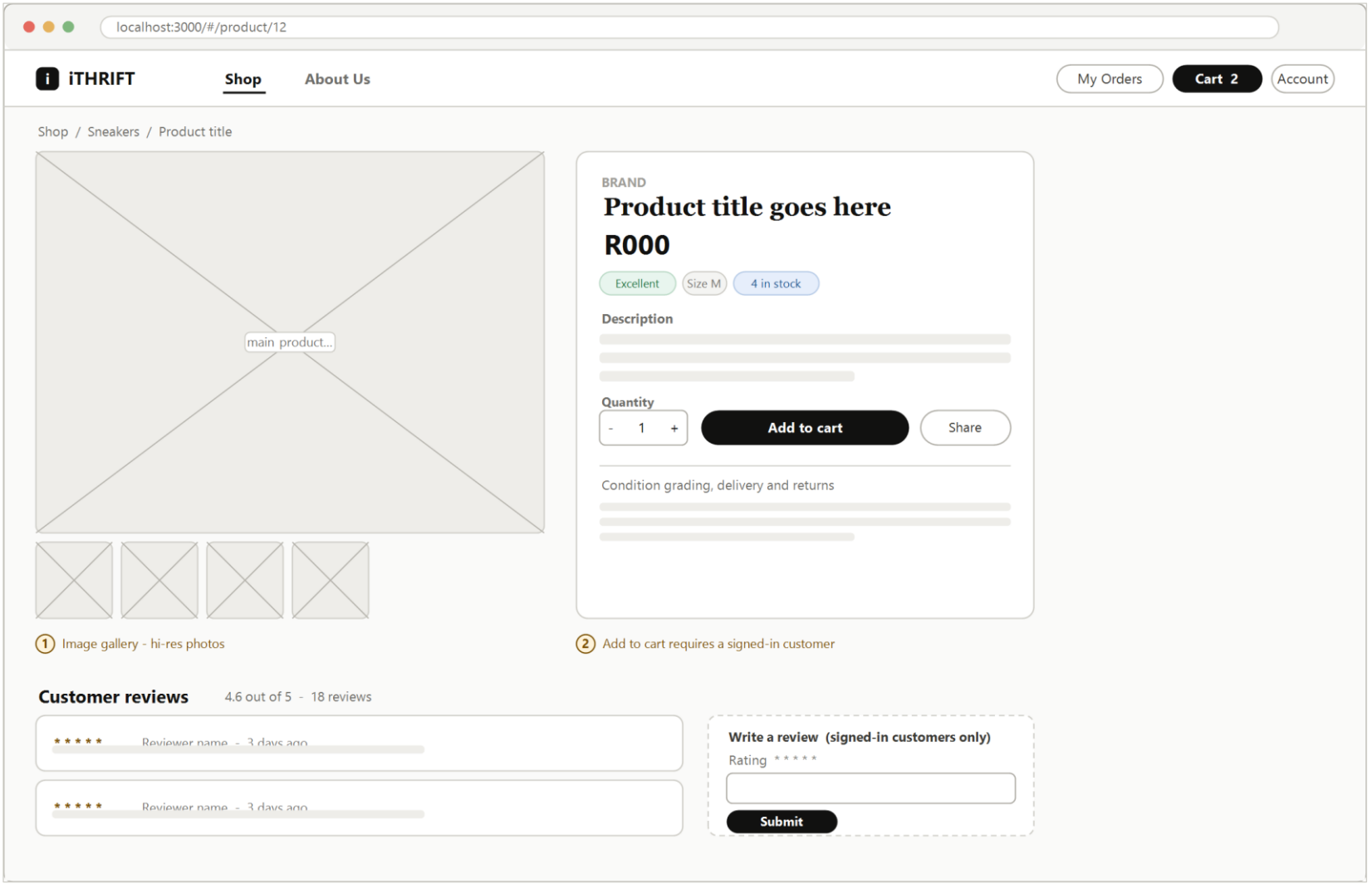



Figure 7: Website: product detail with gallery and reviews (#/product/:id)

- The gallery takes up the left half of the screen. Second-hand stock is one of a kind, so the photographs carry the weight that a size chart carries in ordinary retail.
- Condition, size and stock sit together directly under the price, because those three answer the customer's first question, which is whether this is the right item and whether it is still available.
- Reviews are shown below the fold, and the review form is limited to signed-in customers, which prevents anonymous posts.

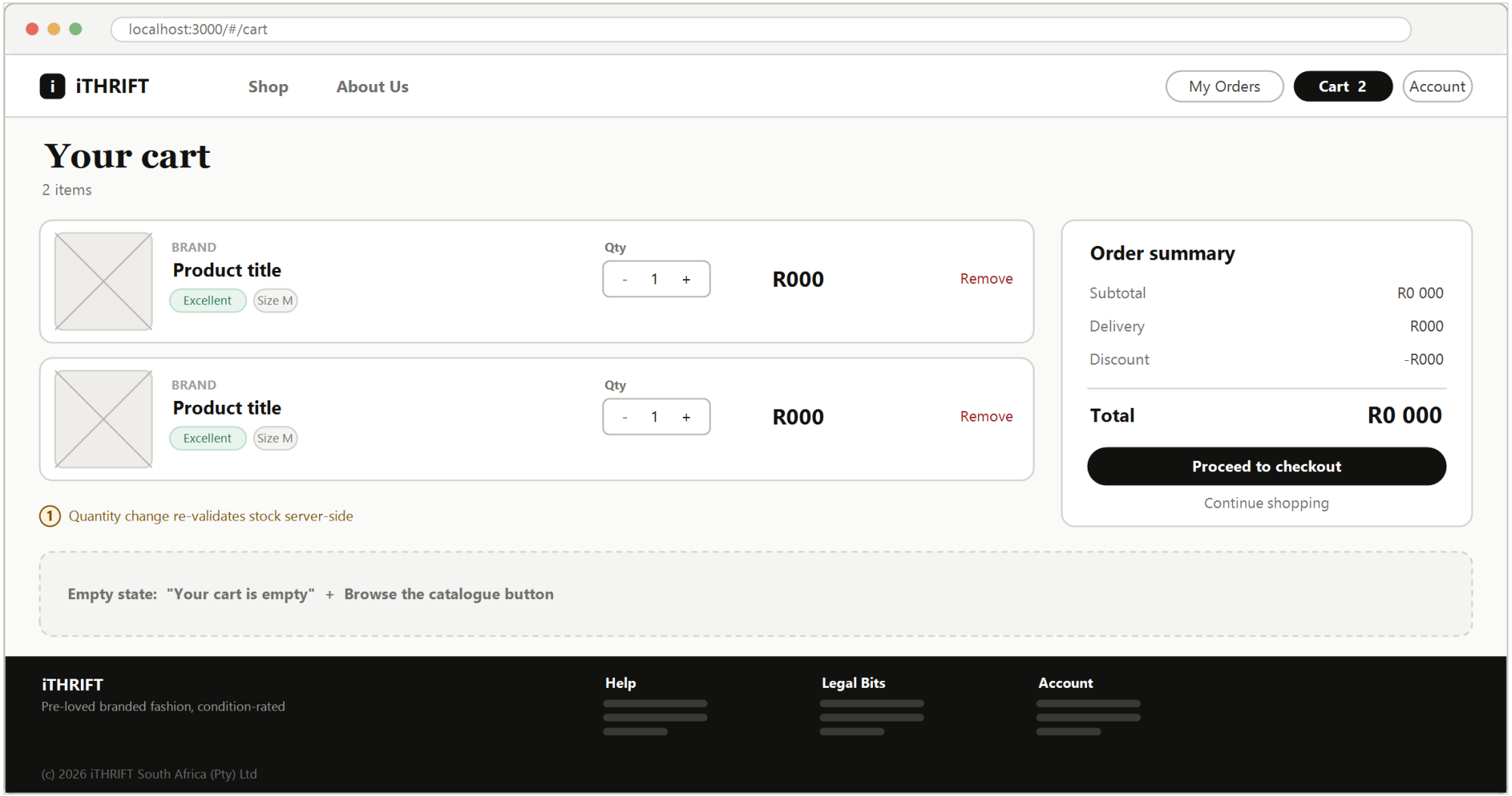



Figure 8: Website: cart with order summary (#/cart)

- The summary panel is fixed to the right of the line items, so the total stays visible while quantities are changed.
- Changing a quantity re-validates the stock on the server. A customer cannot increase a quantity beyond what is available, because the client is not trusted to know the stock level.
- The empty state is specified here rather than left to the build, because an unstyled empty cart is a common way for a prototype to look unfinished.

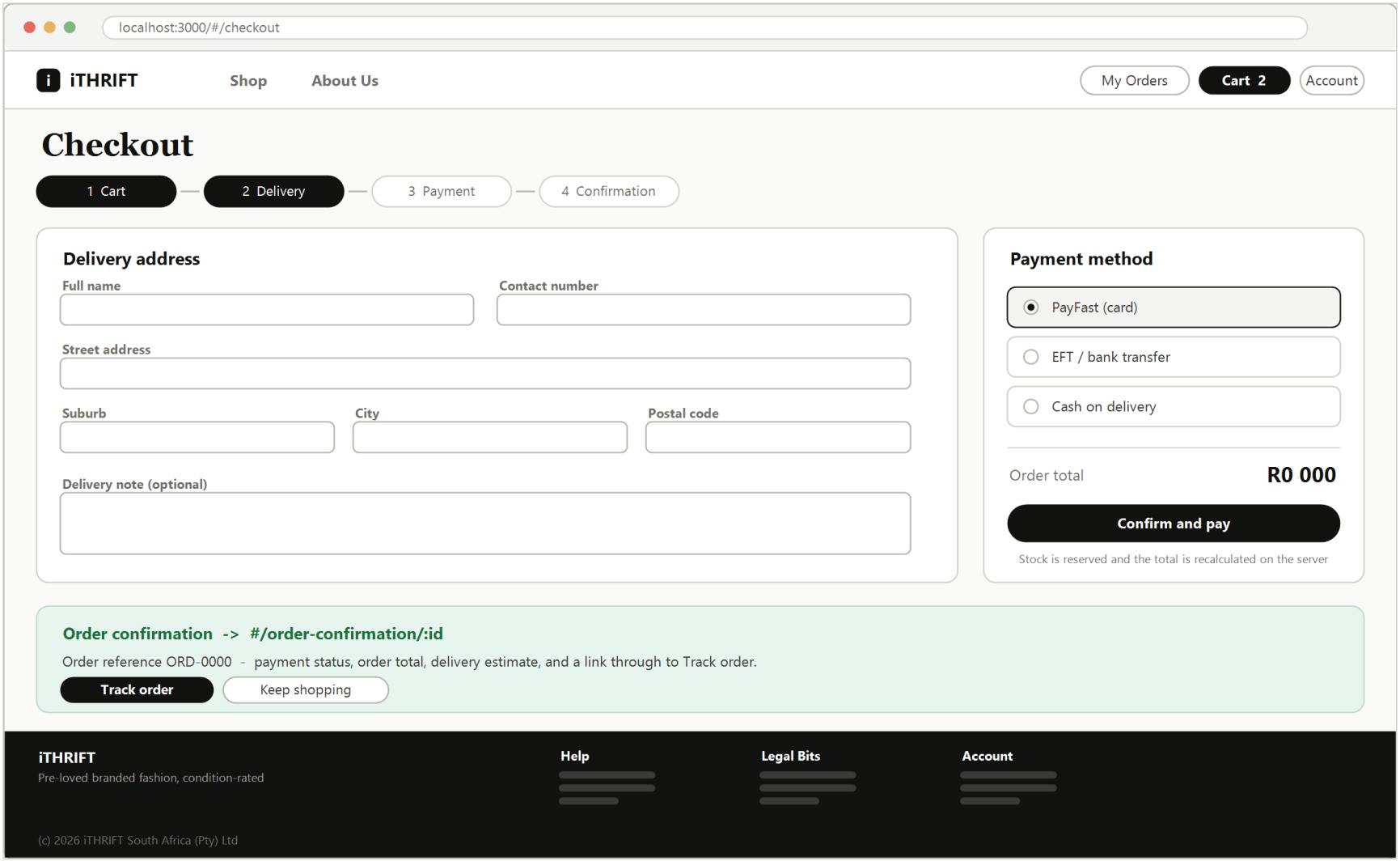



Figure 9: Website: checkout and order confirmation (#/checkout, #/order-confirmation/:id)

- A four-step indicator runs across the top. Showing the customer how many steps remain is a well-established way of reducing abandonment at checkout.
- Three payment methods are offered. Cash on delivery and EFT matter in the South African market, where a meaningful share of customers do not want to enter card details.
- The order total is recalculated on the server when the order is placed, and the stock is locked in the same transaction. This is what stops two customers from buying the same one-off item.
- The confirmation state is drawn on the same sheet because it is the immediate result of the primary action, and the two are best reviewed together.

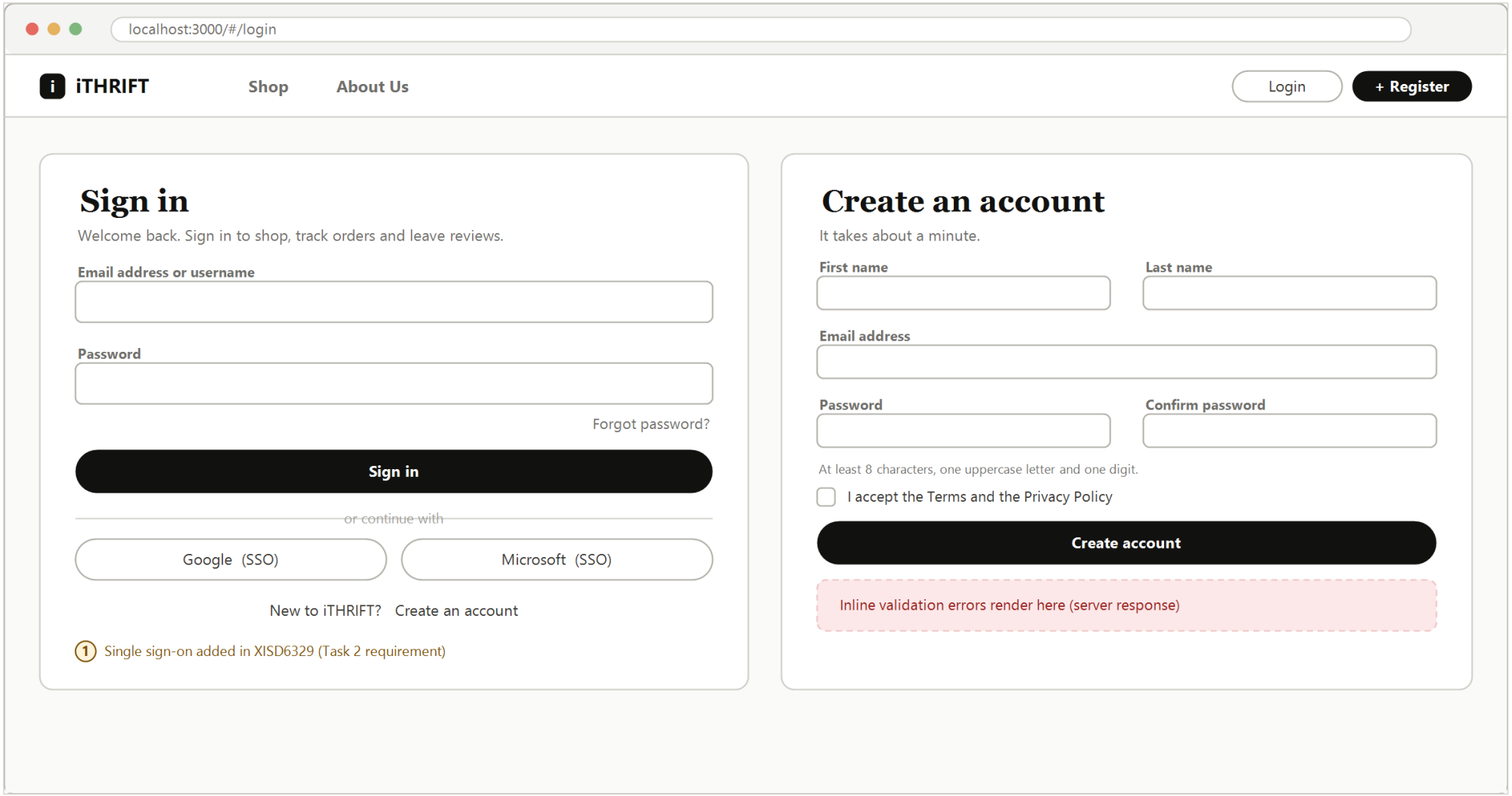



Figure 10: Website: sign in and registration (#/login, #/register)

- Both screens are shown together because they are two halves of one decision, and the link between them must be obvious on each.
- Single sign-on is included here because the module requires it in Task 2. Designing it now avoids having to fit a second button into a finished screen later.
- Password rules are stated next to the field rather than revealed by an error message after submission.
- Validation errors are rendered inline from the server response. Client-side validation is a convenience only, because the server is authoritative.

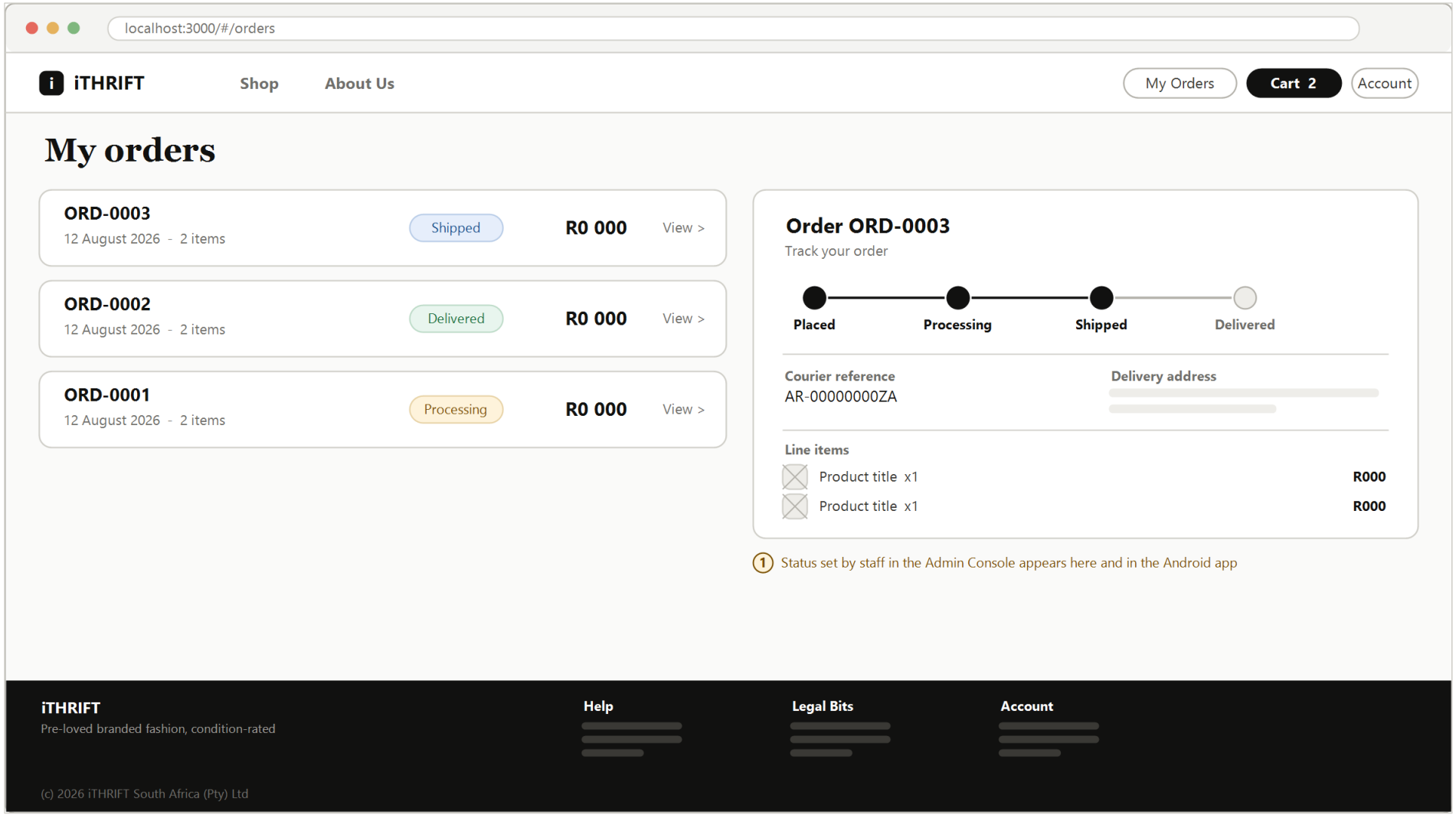



Figure 11: Website: order history and order tracking (#/orders, #/orders/:id)

- The list and the detail are shown side by side, which is how the screen behaves on a wide viewport. On a narrow viewport the detail becomes its own page.
- The status tracker is a horizontal timeline, and the completed stages are filled in. These are the same four stages that staff set in the console, so the customer and the business always see one truth.
- The courier reference appears only once staff have captured it, rather than showing an empty field that looks like a fault.

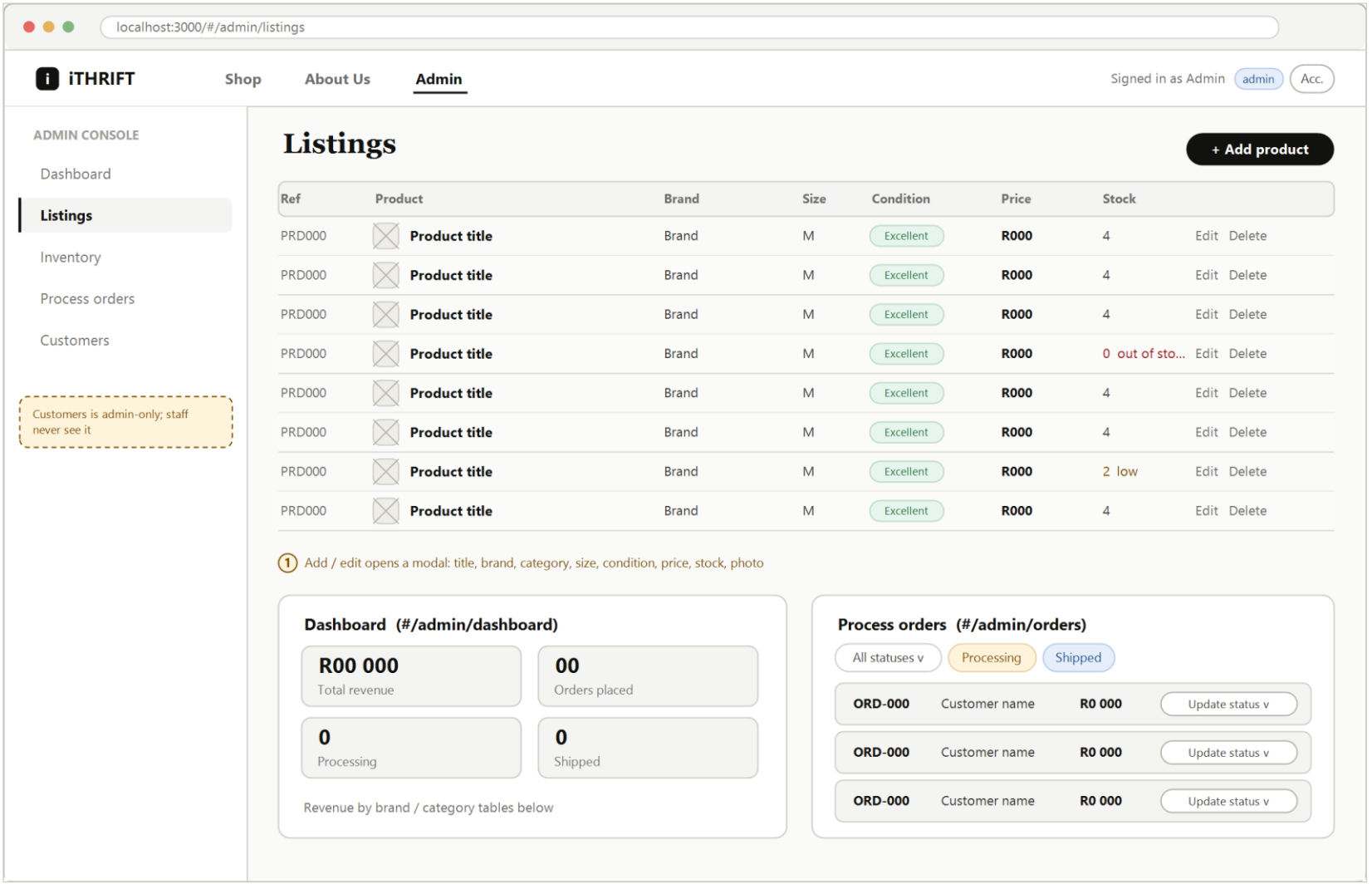



Figure 12: Website: staff and administrator console (#/admin)

- A permanent left sidebar carries the five console sections. Customers is rendered for administrators only. A staff account never sees the item, and the server refuses the request even if the address is typed in by hand.
- Listings is a dense table rather than a card grid, because staff scan and edit rather than browse.
- Stock is colour-coded in the table, with red at zero and amber at three or fewer. The low-stock list on the Inventory screen is generated from the same rule.
- The dashboard and the order-processing panels are drawn at the foot of the sheet, so that all three console sections can be seen in one view.

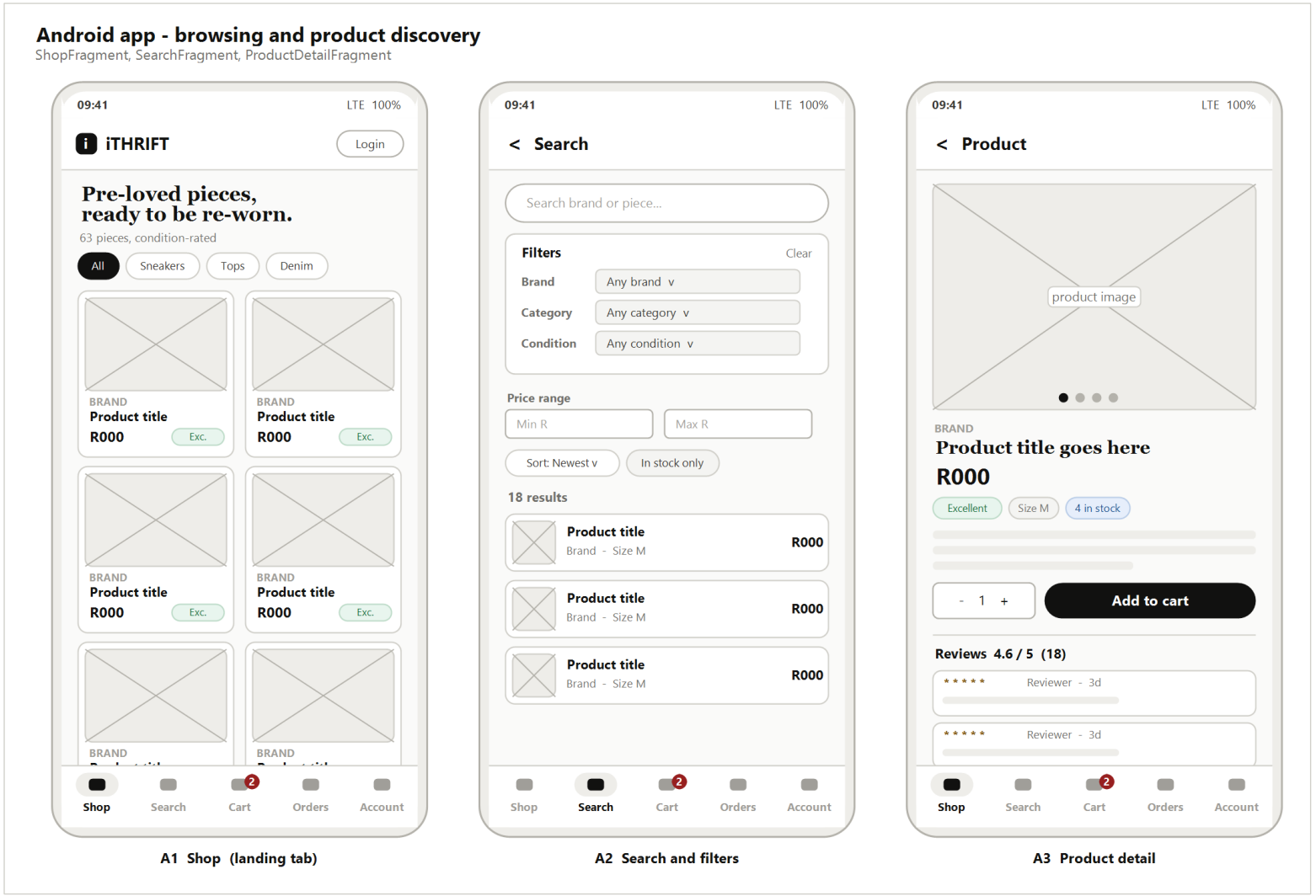



Figure 13: Android application: Shop, Search and Product Detail

- The bottom navigation carries five tabs and is present on every root screen. The Cart tab shows a badge with the item count, so the customer never loses track of what is in the cart.
- The catalogue uses a two-column grid. One column wastes the screen, while three columns make the photographs too small for the customer to judge condition.
- Search is its own tab, with the filters on the screen rather than behind a menu. This is the main structural difference from the website.
- The product detail screen keeps the quantity selector and the add-to-cart button together at a height that is easy to reach with a thumb, rather than at the top of the screen.

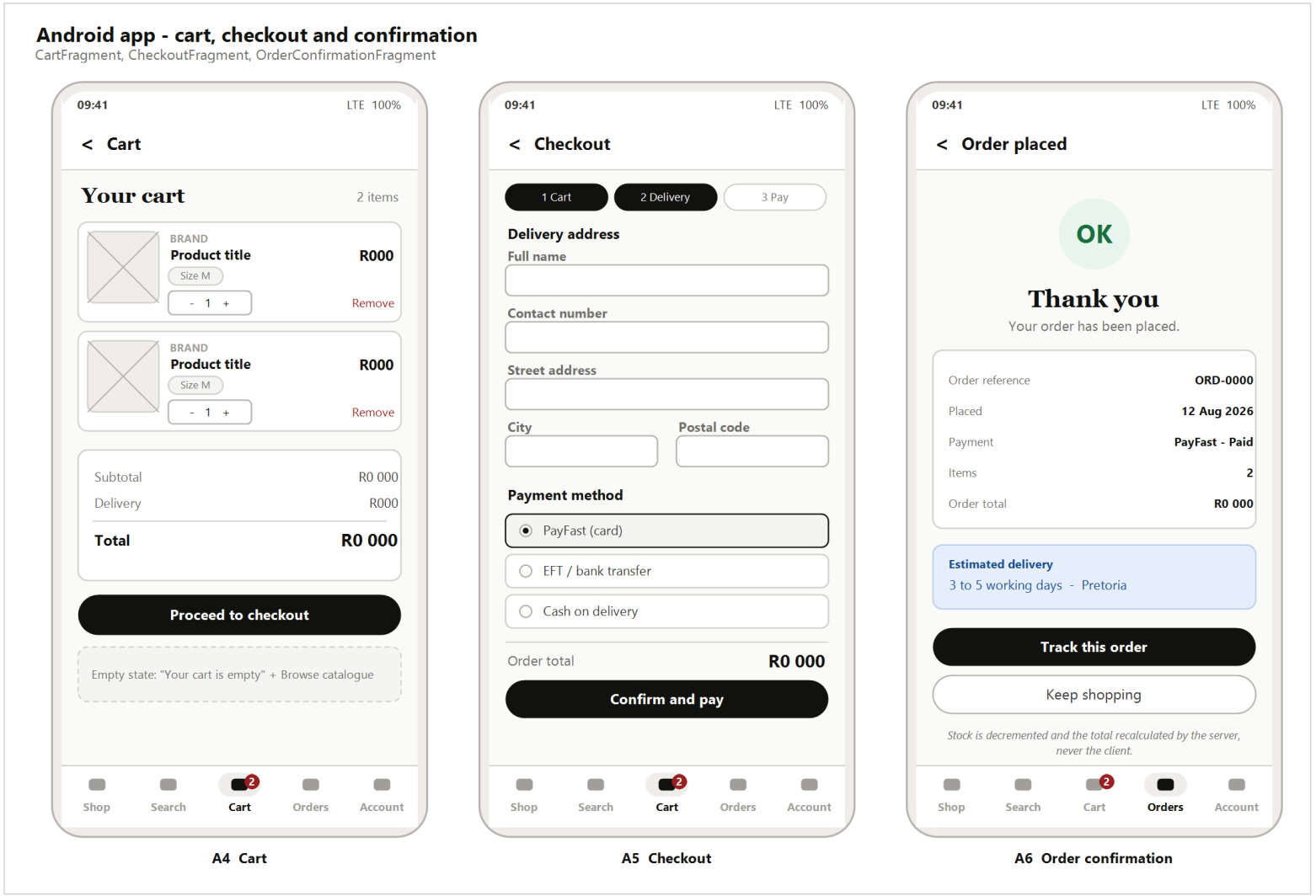



Figure 14: Android application: Cart, Checkout and Order Confirmation

- The cart uses full-width rows rather than the summary panel used on the website, because there is no room for a side panel. The total sits directly above the primary action.
- Checkout is compressed to three steps on the phone, against four on the website, by combining the cart review into the delivery step.
- The confirmation screen leads with the order reference, because that is what the customer will quote if they contact the shop.
- Both Track this order and Keep shopping are offered, because the customer's next intention is genuinely one or the other.

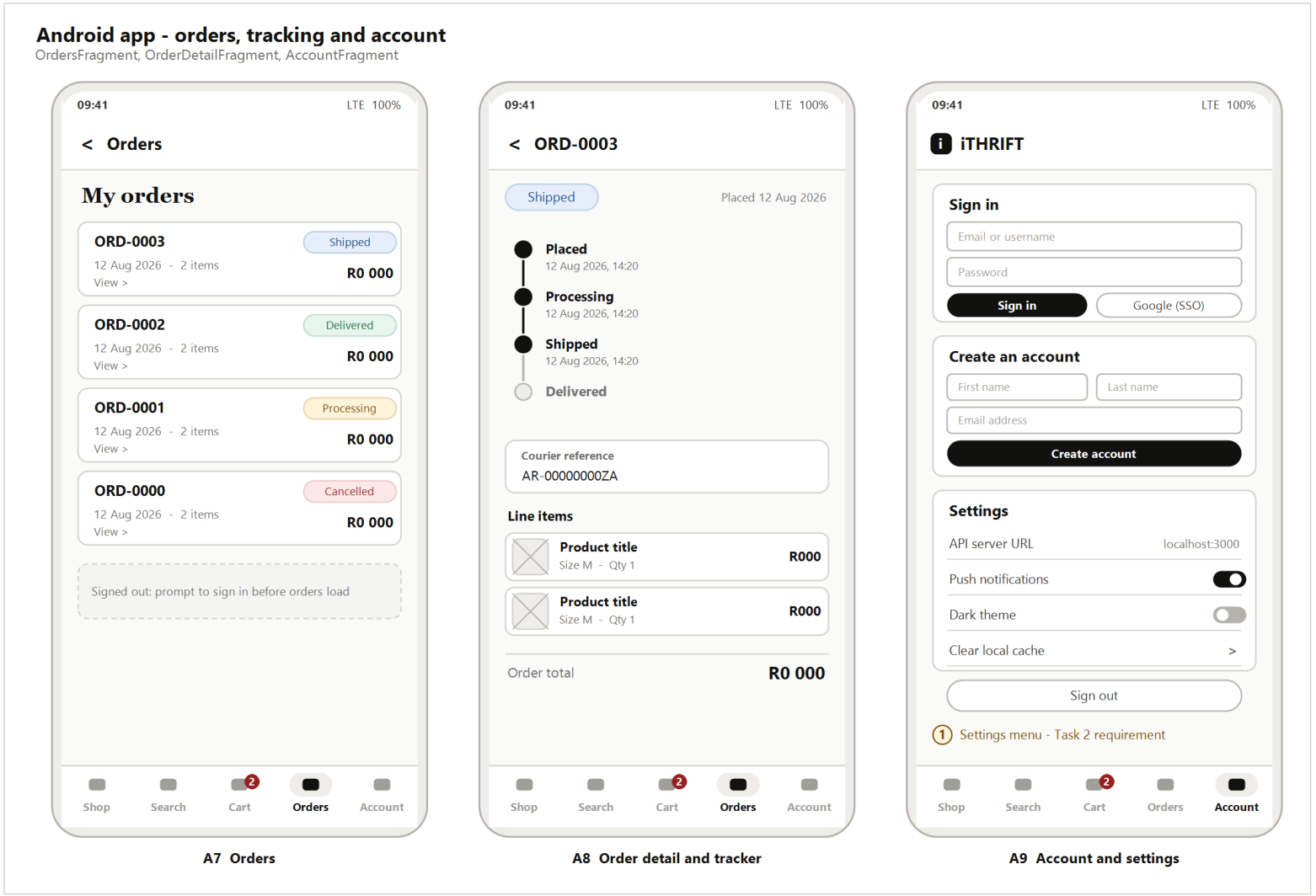



Figure 15: Android application: Orders, Order Detail and Account with settings

- The status tracker is vertical on the phone and horizontal on the website. The stages and their meaning are identical, and only the axis changes.
- Sign-in and registration share the Account tab. A signed-out customer sees both cards, while a signed-in customer sees their profile and the settings list instead.
- The settings screen meets the Task 2 requirement for a settings menu that makes sense for the application, and it covers the API endpoint, notifications, theme and local cache.
- The Orders tab shows a prompt to sign in rather than an empty list when the customer is signed out. This distinguishes "you have no orders" from "we do not know who you are".

## References


- GitHub. 2025. *GitHub Actions documentation*. [Online]. Available at: https://docs.github.com/en/actions [Accessed 8 August 2026].
- Google. 2025. *Guide to app architecture*. [Online]. Available at: https://developer.android.com/topic/architecture [Accessed 8 August 2026].
- Google. 2025. *Material Design 3: Navigation bar*. [Online]. Available at: https://m3.material.io/components/navigation-bar/overview [Accessed 8 August 2026].
- Google. 2025. *Navigation*. [Online]. Available at: https://developer.android.com/guide/navigation [Accessed 8 August 2026].
- Microsoft. 2025. *Azure Boards documentation*. [Online]. Available at: https://learn.microsoft.com/en-us/azure/devops/boards [Accessed 8 August 2026].
- Microsoft. 2025. *What is DevOps?* [Online]. Available at: https://learn.microsoft.com/en-us/devops/what-is-devops [Accessed 8 August 2026].
- Nielsen, J. 1994. *10 usability heuristics for user interface design*. [Online]. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/ [Accessed 8 August 2026].
- The Independent Institute of Education. 2026. *Module manual XISD6329: Work Integrated Learning 3B*. Johannesburg: The Independent Institute of Education.
- World Wide Web Consortium. 2023. *Web Content Accessibility Guidelines (WCAG) 2.2*. [Online]. Available at: https://www.w3.org/TR/WCAG22/ [Accessed 8 August 2026].